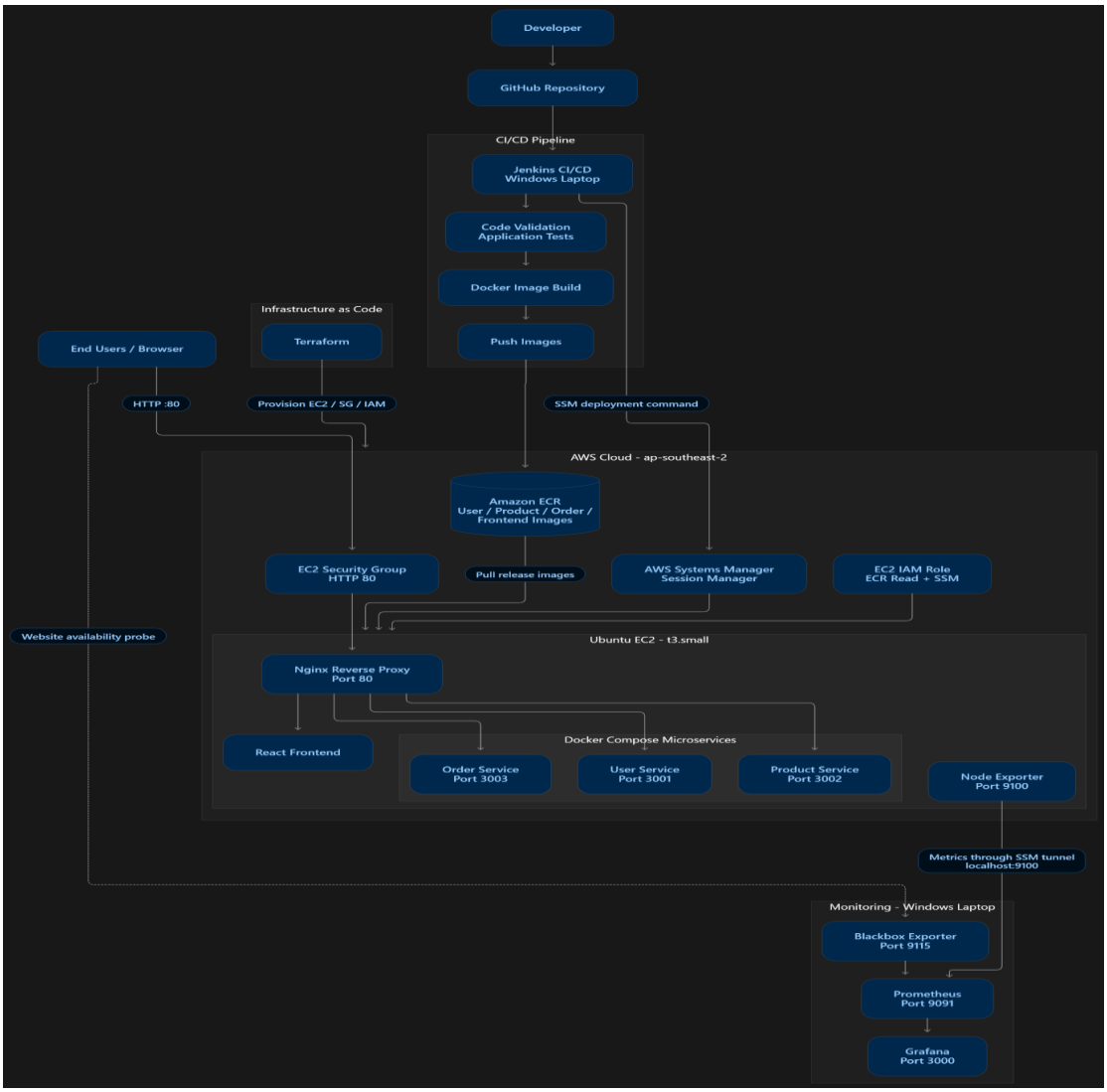
SHOPSPHERE

Microservices Application Deployment & DevOps Implementation

AWS • Terraform • Docker • Jenkins CI/CD • Prometheus • Grafana • Blackbox Exporter

Submitted by:- Shreyas N
shreyasn@thestackly.com

Infrastructure Diagram



ShopSphere end-to-end deployment and monitoring architecture.

1. Project at a Glance

A practical overview of what was built and how the pieces work together.

ShopSphere is a web-based shopping application built as a small microservices system. The project was used to practise a realistic DevOps delivery lifecycle: take application source code, package it, publish versioned images, deploy it to cloud infrastructure, and monitor the running service.

The application was deployed to one Ubuntu EC2 instance in AWS. Docker Compose runs the frontend, three backend services and an Nginx reverse proxy. Jenkins automates the build and release workflow. Terraform describes the cloud infrastructure as code. AWS Systems Manager provides remote access and deployment without opening SSH to the internet.

For monitoring, Node Exporter exposes host metrics, Prometheus collects them, Grafana displays dashboards and evaluates alerts, and Blackbox Exporter checks whether the public ShopSphere website responds successfully.

Item	Project implementation
Application	ShopSphere e-commerce web application
Runtime	AWS EC2, Ubuntu 24.04, t3.small
Region	ap-southeast-2 (Sydney)
Application packaging	Docker images and Docker Compose
Ingress / reverse proxy	Nginx on HTTP port 80
CI/CD	Jenkins on Windows workstation
Infrastructure as Code	Terraform
Container registry	Amazon ECR
Remote deployment	AWS Systems Manager (SSM)
Monitoring	Node Exporter, Prometheus, Grafana, Blackbox Exporter

In plain language The goal was not simply to make the website run. The goal was to make deployment repeatable, recoverable and observable.

2. Business and Engineering Objectives

A DevOps implementation should solve operational problems, not just introduce tools. The project objectives were selected to demonstrate the most important stages of a software delivery lifecycle.

Objective	Why it matters	How it was addressed
Repeatable releases	Reduce manual deployment errors.	Jenkins pipeline creates build-number tags and deploys through SSM.
Consistent runtime	Avoid differences between laptop and server environments.	Application components are packaged as Docker images.
Infrastructure repeatability	Make cloud resources reviewable and reproducible.	Terraform provisions EC2, security group and IAM resources.
Secure administration	Avoid exposing SSH and server metrics publicly.	SSM Session Manager and port forwarding are used.
Service recovery	Recover when a release cannot be deployed.	Deployment script attempts rollback to the previous image tag.
Operational visibility	Know whether the host and website are responding.	Prometheus, Grafana and Blackbox Exporter provide metrics and probes.

The scope was intentionally cost-conscious. A single EC2 instance and Docker Compose were selected instead of a Kubernetes cluster or a multi-instance architecture. This keeps the learning environment manageable while still demonstrating core DevOps practices.

3. Application Overview – ShopSphere

ShopSphere provides a shopping experience where a visitor can browse products, create an account, log in, select a product, add it to a cart and place an order. The supplied project evidence shows the main user journey through the browser.

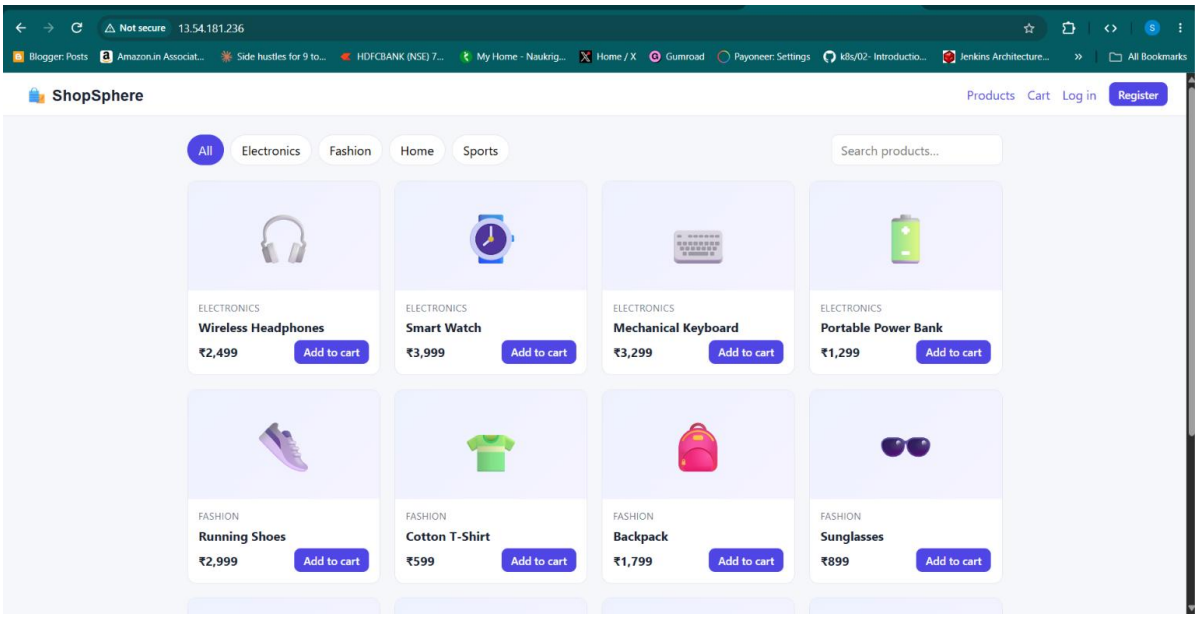


Figure 1. ShopSphere storefront showing product categories and product cards.

The user-facing frontend is separated from the backend business functions. This separation helps the project demonstrate how independently packaged services communicate through a reverse proxy.

Application function	Observed user experience
Product browsing	The storefront displays product cards and category navigation.
Registration	A visitor can open the create-account form and submit user details.
Login	A registered user can access the application using the login flow.
Product selection	A visitor can open a product detail view.
Cart	A selected product can be added to the cart and viewed in the cart.
Order placement	The user can submit the cart and see an order confirmation message.

4. Application User Journey – Registration and Login

The first stage of the shopping journey is account access. The supplied screenshots show a registration form with name, email and password fields, followed by the login navigation on the storefront.

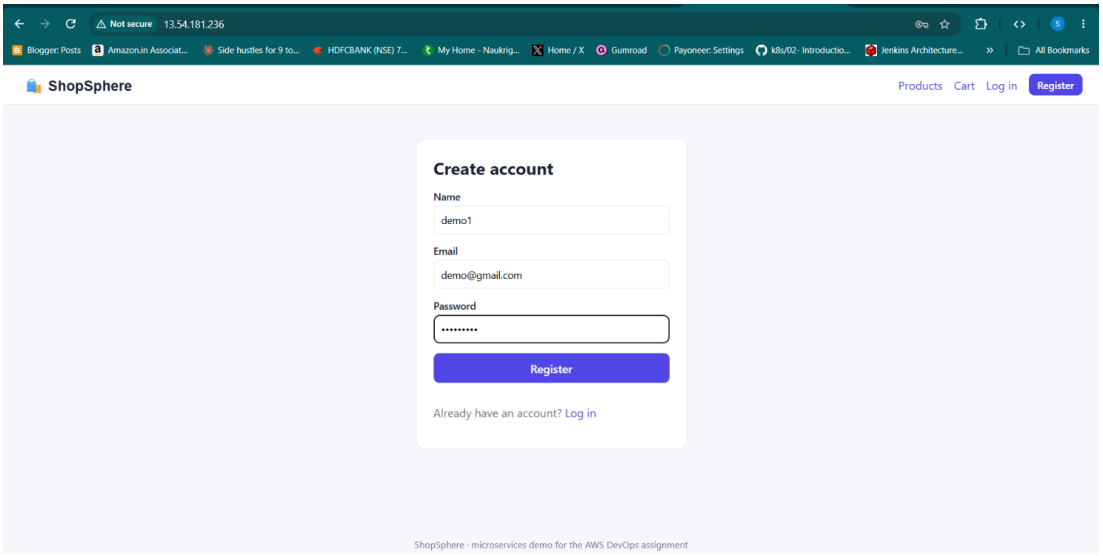


Figure 2. User registration page.

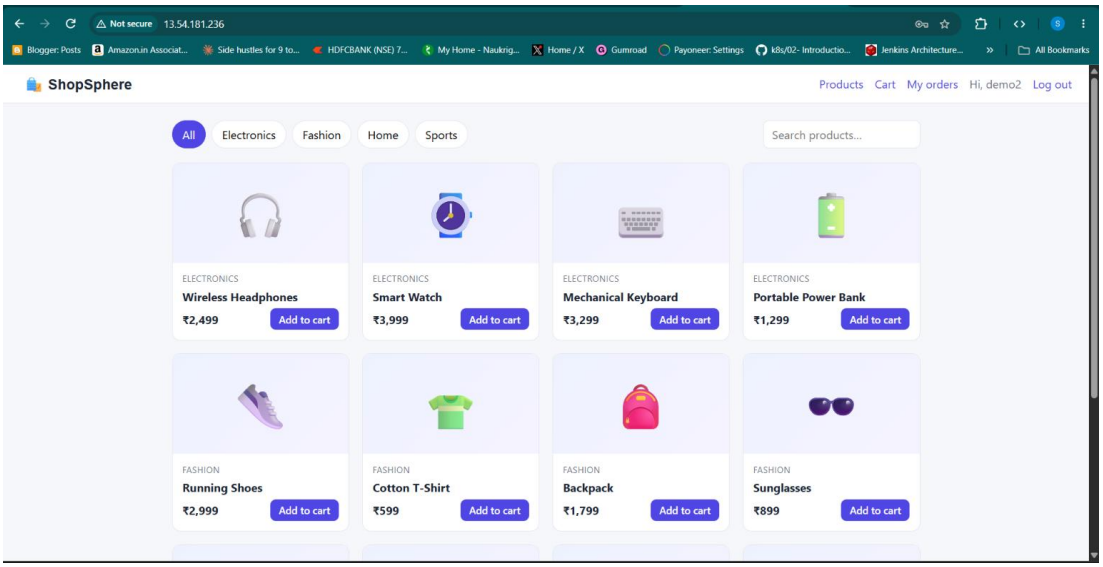


Figure 3. ShopSphere storefront after the user login flow.

From a DevOps perspective, these screens are useful validation evidence because they demonstrate that the frontend is not merely returning an empty HTTP response. The deployed web application renders its real interface.

Security reminder Passwords and authentication tokens should be handled securely by the application. Never place real credentials, JWT secrets or production .env values in screenshots, Git commits or this report.

5. Application User Journey – Products, Cart and Orders

The product flow demonstrates the interaction between the frontend and product/order backend services. A user can browse categories, open a product, add it to the cart, review the cart and place an order.

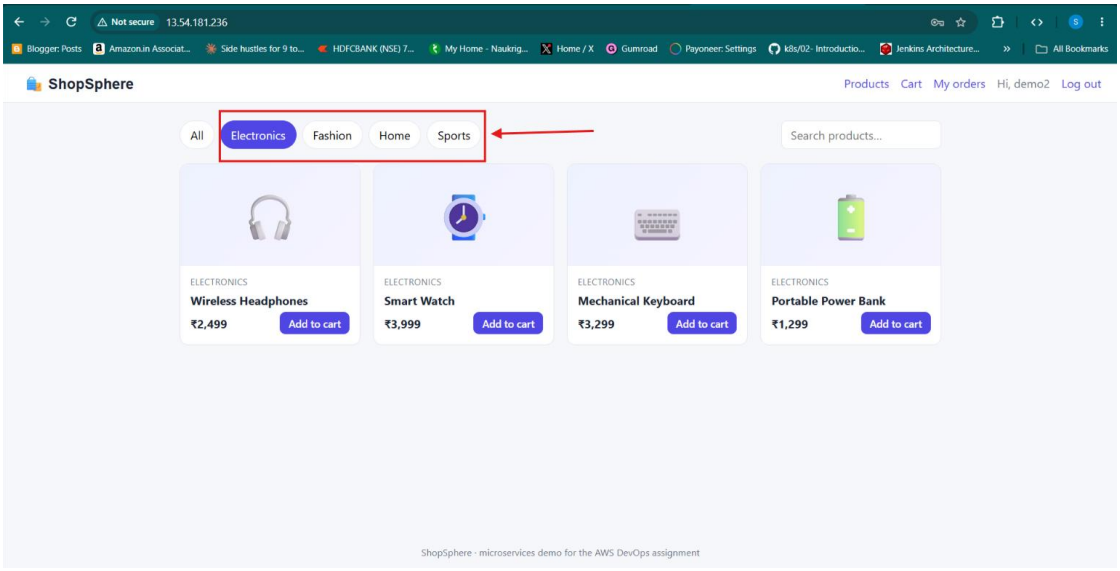


Figure 4. Product categories and product selection.

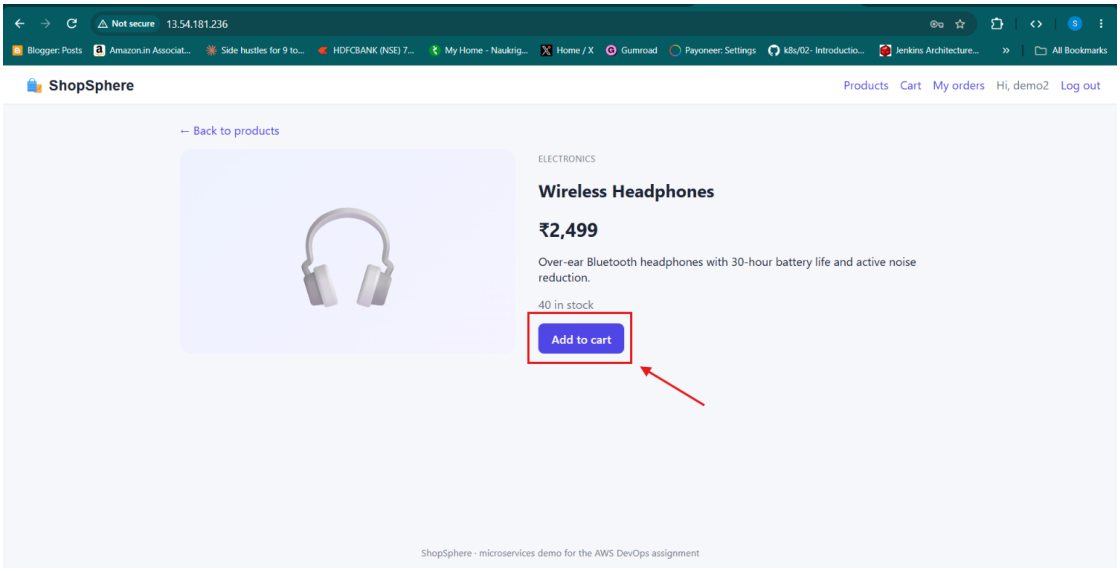


Figure 5. Product detail view and add-to-cart action.

6. Application User Journey – Cart and Order Confirmation

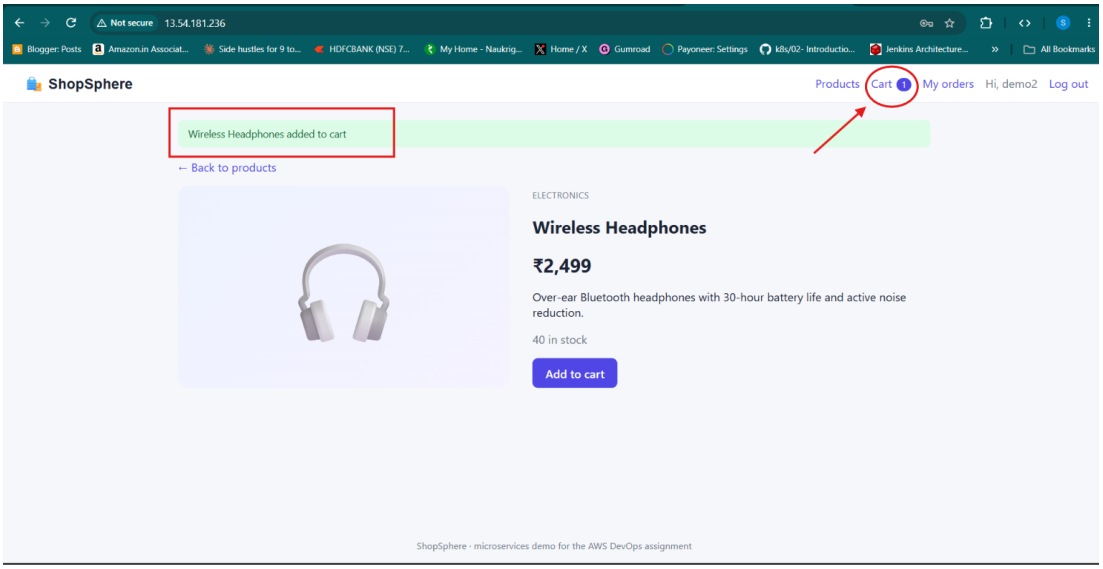


Figure 6. Confirmation that a product was added to the cart.

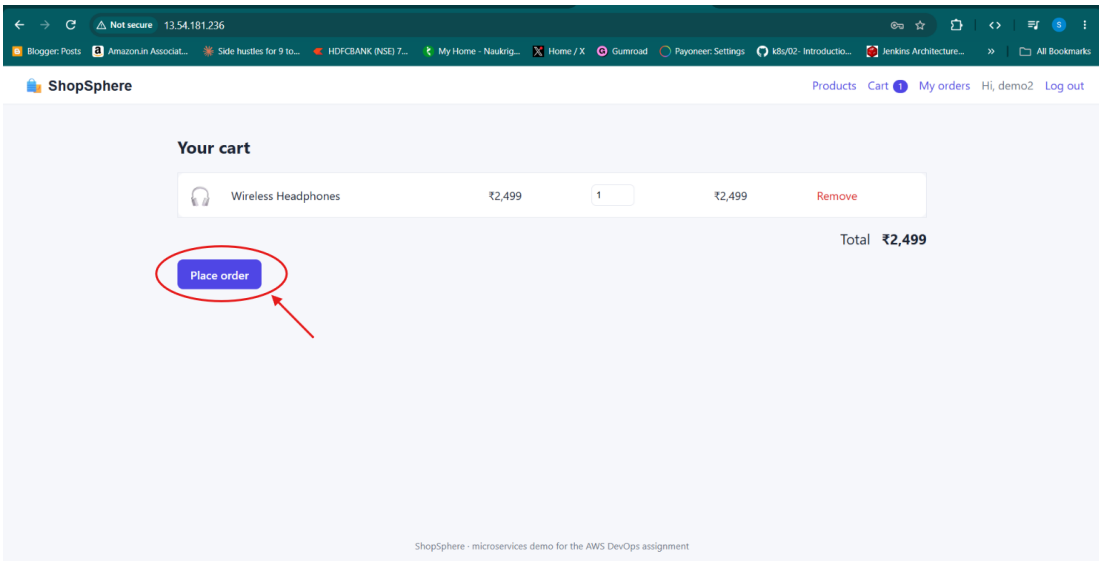


Figure 7. Cart view with the place-order action.

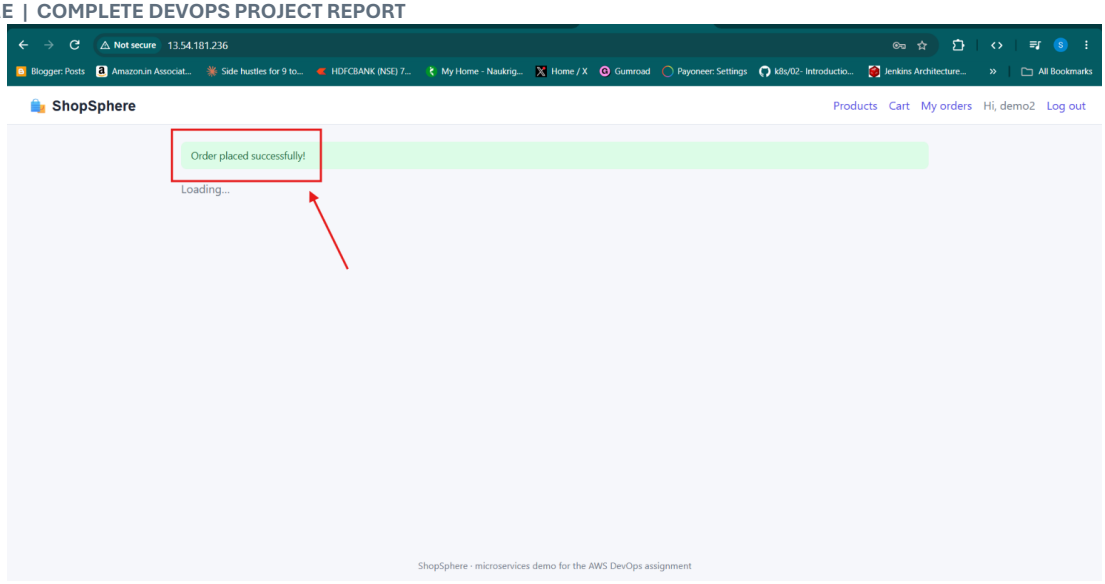


Figure 8. Order placement confirmation displayed by ShopSphere.

These screenshots are application-level evidence. They complement infrastructure checks such as container health and HTTP status codes.

7. Microservices and Request Routing

The deployed Compose stack contains five containers: frontend, user-service, product-service, order-service and Nginx. The backend services listen on their internal application ports. Nginx is the public entry point and forwards requests to the appropriate service.

Component	Container port	Role
Frontend	80	Serves the ShopSphere user interface.
User service	3001	User registration and login-related API operations.
Product service	3002	Product listing and product-related operations.
Order service	3003	Cart/order-related operations.
Nginx reverse proxy	80	Receives browser traffic and routes requests internally.

```
# Nginx routing concept (configured in nginx.conf)
 /                -> http://frontend:80
 /api/users/      -> http://user-service:3001
 /api/products/   -> http://product-service:3002
 /api/orders/     -> http://order-service:3003
```

Docker Compose provides a private bridge network so containers can resolve one another by service name. The backend services are not required to expose their application ports directly to the public internet.

Why a reverse proxy? The browser uses one public entry point. Nginx routes requests internally, which simplifies the public network surface and avoids exposing every microservice separately.

8. Technology Stack – What We Used and Why

Technology	Where it was used	Why it was selected
AWS EC2	Runs the application host.	Provides a controllable Linux server in the cloud.
Ubuntu 24.04	Operating system on EC2.	Common Linux server environment and good Docker support.
Terraform	Infrastructure definition.	Makes cloud resources repeatable and reviewable.
AWS IAM	Access control for Jenkins and EC2.	Separates user and machine permissions.
AWS SSM	Remote access and deployment commands.	Avoids requiring inbound SSH for normal operations.
Amazon ECR	Stores versioned application images.	Private AWS container registry integrated with IAM.
Docker Engine	Runs containers on EC2.	Packages services and provides isolated runtime environments.
Docker Compose	Coordinates five containers.	Simple orchestration for a single-host deployment.
Nginx	Public reverse proxy.	Routes incoming HTTP requests to the correct service.
Jenkins	CI/CD automation on Windows.	Automates validation, image build, push and deployment.
Git/GitHub	Source control and repository hosting.	Tracks application and infrastructure changes.
Node Exporter	EC2 host metrics endpoint.	Exposes Linux CPU, memory, filesystem and network metrics.
Prometheus	Metric collection on Windows.	Scrapes and stores time-series metrics.
Grafana	Dashboards and alert evaluation.	Visualizes Prometheus data and evaluates alert rules.
Blackbox Exporter	Public website probe on Windows.	Checks HTTP availability and response characteristics.

9. AWS Infrastructure – Compute and Network

The application host was created in AWS in the Sydney region (ap-southeast-2). The recorded instance was an Ubuntu 24.04 x86_64 t3.small instance with an encrypted 20 GB gp3 root volume.

Setting	Recorded value
AWS Region	ap-southeast-2
EC2 instance ID	i-0af087445cc5f5878
Instance type	t3.small
Operating system	Ubuntu 24.04
Root volume	20 GB gp3, encrypted
Deployment path	/opt/shopsphere
Public application port	TCP 80
SSH inbound	Not included in the described Terraform security group

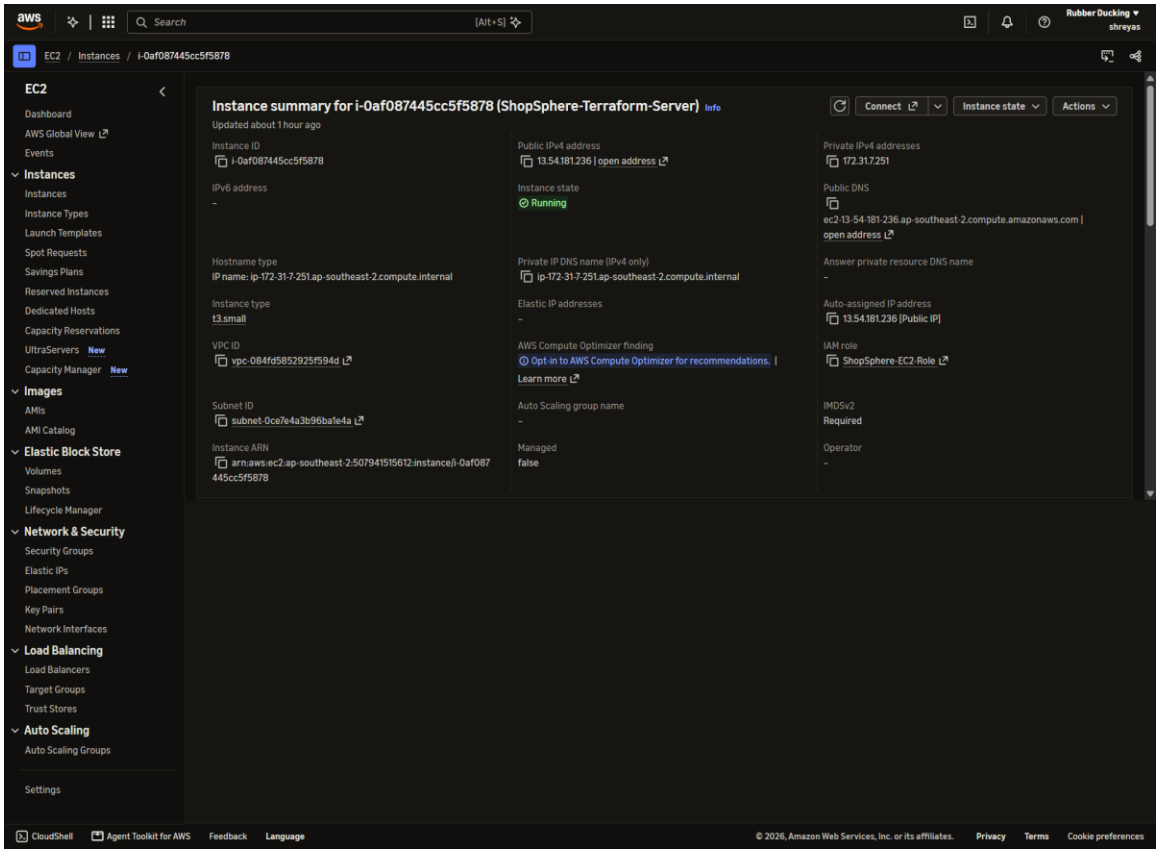


Figure 9. AWS EC2 instance details used to host ShopSphere.

The Terraform-managed security group allowed inbound TCP port 80 from 0.0.0.0/0 and allowed outbound traffic. Administrative access was handled through Systems Manager. The instance public IP changed after stop/start, so operational instructions should always use the current address.

Production note For a public production site, add HTTPS, DNS, appropriate access restrictions and an availability design. A public IPv4 address alone is not a production ingress strategy.

10. IAM Roles, SSM and Secure Access

The implementation separated AWS identities by responsibility. The EC2 instance role gave the host permission to pull private ECR images and register with Systems Manager. Jenkins used a dedicated IAM user credential for ECR publishing and deployment commands. A separate terraform CLI profile was used for administrative tasks and interactive SSM access.

Identity / role	Purpose
ShopSphere-EC2-Role	Runtime access from EC2 to ECR and SSM.
ShopSphere-EC2-Profile	Attaches the EC2 role to the instance.
jenkins-shopsphere	Jenkins access for ECR image publishing and SSM deployment calls.
terraform CLI profile	Terraform and operator CLI access used in the project.

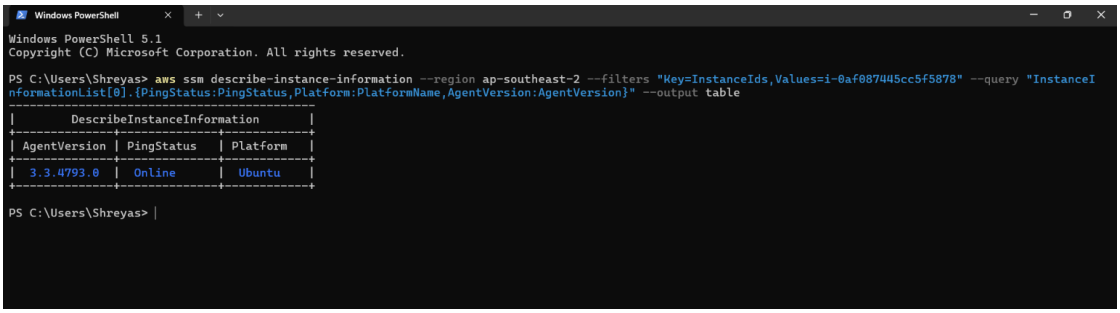


Figure 10. Systems Manager reports the EC2 managed instance as Online.



Figure 11. Interactive Ubuntu session opened through SSM Session Manager.

```
# Windows PowerShell – connect to the EC2 instance through SSM
aws ssm start-session --target i-0af087445cc5f5878 --profile terraform --region ap-southeast-2
```

Security principle The EC2 host should not store long-lived IAM access keys. Use its attached instance role for AWS calls from the server.

11. Linux Host Preparation and Docker Runtime

After connecting to Ubuntu through SSM, the host was checked to confirm Docker was installed, enabled and running. The application directory was then inspected to confirm the Compose configuration, release tag file and Nginx configuration were present.

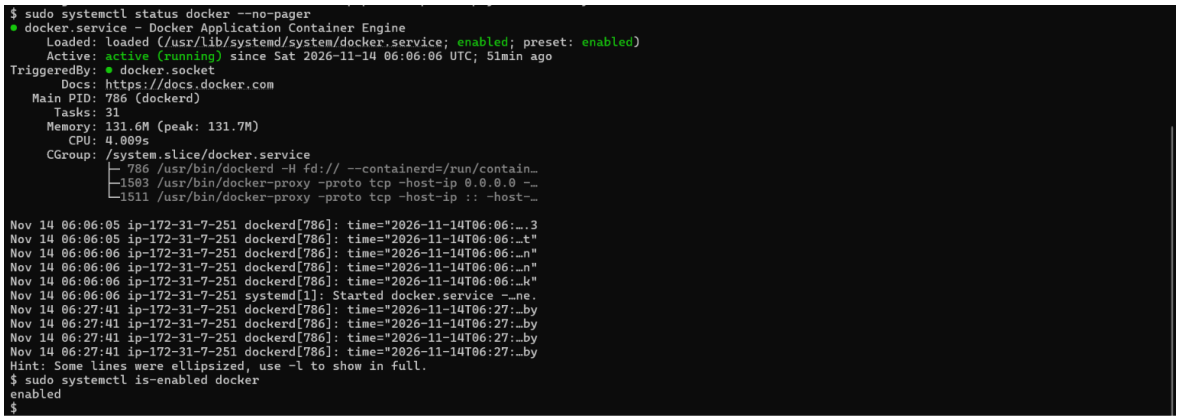


Figure 12. Docker service status and startup configuration on Ubuntu.



Figure 13. ShopSphere deployment directory on EC2.

```
# Ubuntu EC2 – inspect Docker and deployment directory
sudo systemctl status docker --no-pager
sudo systemctl enable docker
cd /opt/shopsphere
ls -la
```

Docker Engine and Docker Compose were used on the EC2 host. The application deployment directory contains docker-compose.yml, image-tag.env and the Nginx configuration directory. The image-tag.env file records which release tag should be deployed.

12. Docker Compose – Running the Five Containers

Docker Compose brings up the frontend, three backend services and Nginx. Health checks were configured for backend services. This provides a simple way to inspect service state and detect a backend that has not yet become healthy.

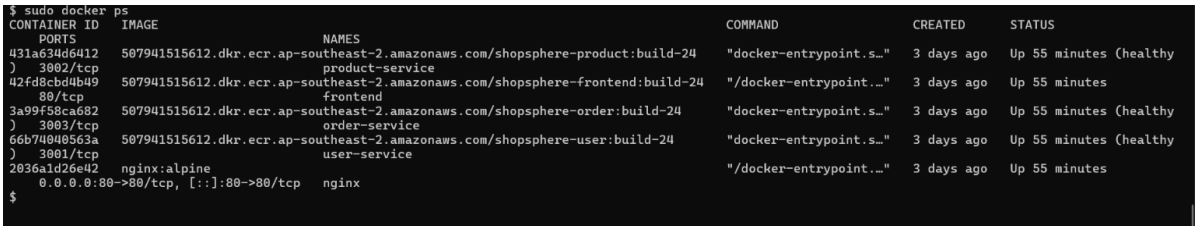


Figure 14. All five ShopSphere containers visible in the Compose status output.

```
# Ubuntu EC2 – inspect Compose services
cd /opt/shopsphere
cat image-tag.env
docker compose --env-file image-tag.env ps

# Validate the Compose configuration
docker compose --env-file image-tag.env config -q
```

The known-good release recorded during the project was build-24. At the verification checkpoint, frontend and Nginx were running, while user, product and order services were reported healthy.

What health checks tell us A running container only confirms that its process is active. A healthy status adds an application-level check, such as a successful request to the service health endpoint.

13. Application Verification on the EC2 Host

A local HTTP request was used to verify that Nginx could serve the application from the EC2 host. The recorded response was HTTP 200. The browser screenshots in this report then verified that the application UI and user journey were rendered.



Figure 15. Local HTTP verification returning status 200.

```
# Ubuntu EC2 – check the local website response
curl -sS -o /dev/null -w 'HTTP Status: %{http_code}\n' http://localhost/

# Optional: inspect recent service logs
docker compose --env-file image-tag.env logs --tail=100
```

A 200 response is a useful first check, but it does not prove every API path or business transaction works. The browser journey and service health checks provide additional evidence.

14. Infrastructure as Code – Terraform

Terraform was used to describe and provision the core AWS resources. The project files included provider configuration, variables, main resource definitions, IAM, security groups, ECR references and outputs.

Terraform file	Responsibility
providers.tf	AWS provider and region configuration.
variables.tf	Reusable inputs such as region, instance type and naming.
main.tf	Core EC2 and related infrastructure resources.
iam.tf	IAM role, policies and instance profile.
security-groups.tf	Network access rules for the instance.
ecr.tf	References to existing ECR repositories.
outputs.tf	Useful deployment outputs such as instance details.

Command	What it does	Project result
terraform init	Downloads providers and initializes working directory.	Completed.
terraform validate	Checks Terraform configuration syntax and structure.	Completed.
terraform plan -out=tfplan	Previews changes and saves a plan.	6 add, 0 change, 0 destroy.
terraform apply tfplan	Applies the saved plan.	Completed.
terraform output	Displays configured outputs.	Used.

State-file safety The local Terraform state and backup files must not be committed to Git. The lock file should be retained. A team environment should use a secured remote backend with locking.

15. Amazon ECR – Image Storage and Release Tags

Amazon ECR stores the versioned Docker images used by the EC2 deployment. Four repositories were used: one each for user-service, product-service, order-service and frontend. The Jenkins pipeline tags images with a build number so a release can be identified and rolled back.

Repository	Image purpose
shopsphere-user	User service image
shopsphere-product	Product service image
shopsphere-order	Order service image
shopsphere-frontend	Frontend image

```
# Windows Jenkins pipeline – ECR login pattern
aws ecr get-login-password --region %AWS_REGION% | docker login --username AWS --password-stdin %ECR_REGISTRY%

# Release tag pattern used by the pipeline
IMAGE_TAG=build-${BUILD_NUMBER}
```

The EC2 instance role has ECR read access so it can pull images during deployment. Jenkins credentials have ECR push permissions so the build agent can publish new images.

Release discipline Keep a known-good previous image tag until the new release is verified. Avoid using only the mutable latest tag for rollback-sensitive deployments.

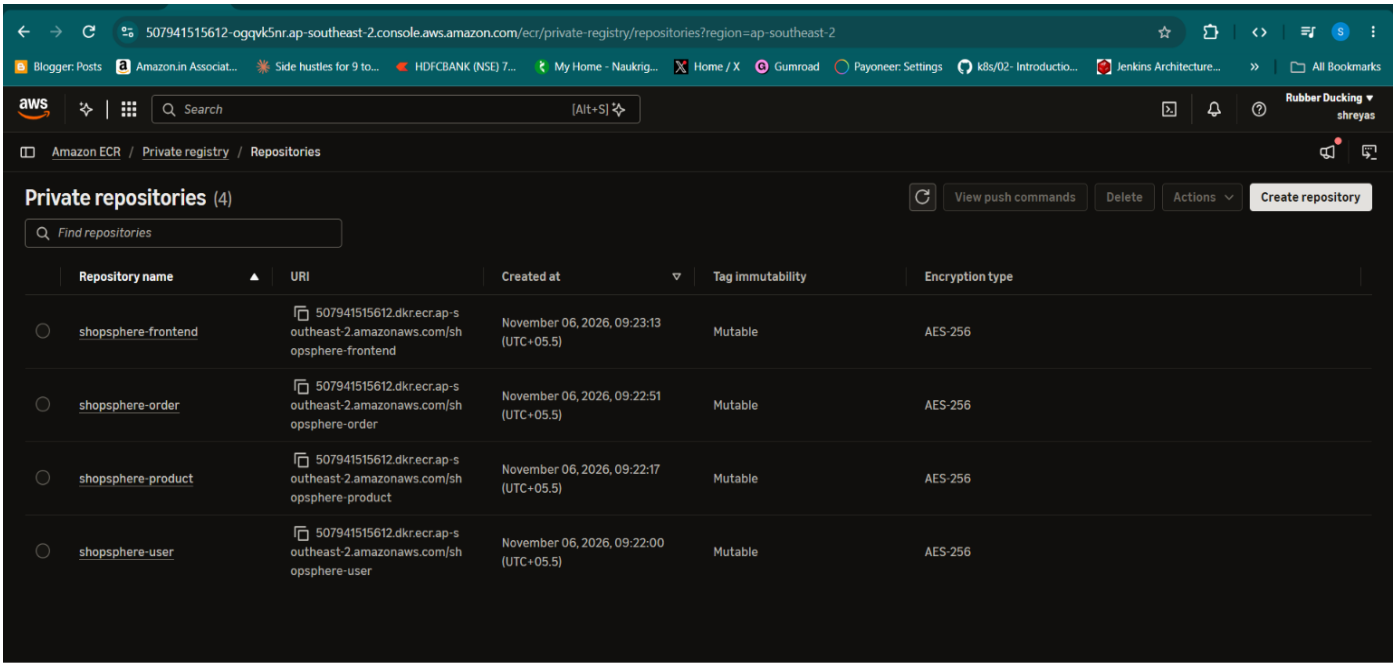


Figure 16. Jenkins CI/CD build process.

16. Jenkins CI/CD – End-to-End Pipeline

Jenkins ran on the Windows workstation. The job E-Commerce-CI-CD automated the path from source checkout to deployment. The pipeline used a Jenkins credential ID rather than hardcoding AWS credentials in the Jenkinsfile.

Pipeline stage	Action
Source Checkout	Fetch source from GitHub.
Set Release Version	Generate build-${BUILD_NUMBER}.
Prepare Environment	Create local .env from .env.example if missing.
Configuration Validation	Run docker compose config -q.
Application Tests	Run npm ci and npm test --if-present where configured.
Build Docker Images	Build four images using ECR repository names and release tag.
Verify Docker Images	Check the expected local images.
Authenticate and Push	Log in to ECR and push all four images.
Deploy to EC2	Call scripts/deploy-ssm.ps1 to execute deployment through SSM.

The pipeline initially encountered ECR permission and Docker proxy issues. The permissions were corrected and Docker Desktop proxy configuration was reviewed. Subsequent pipeline runs were reported successful, including deployment through SSM.

Webhook note The documented evidence confirms the pipeline stages and successful manual/job executions. A GitHub webhook trigger should only be described as complete if the webhook configuration and automatic trigger were separately tested.

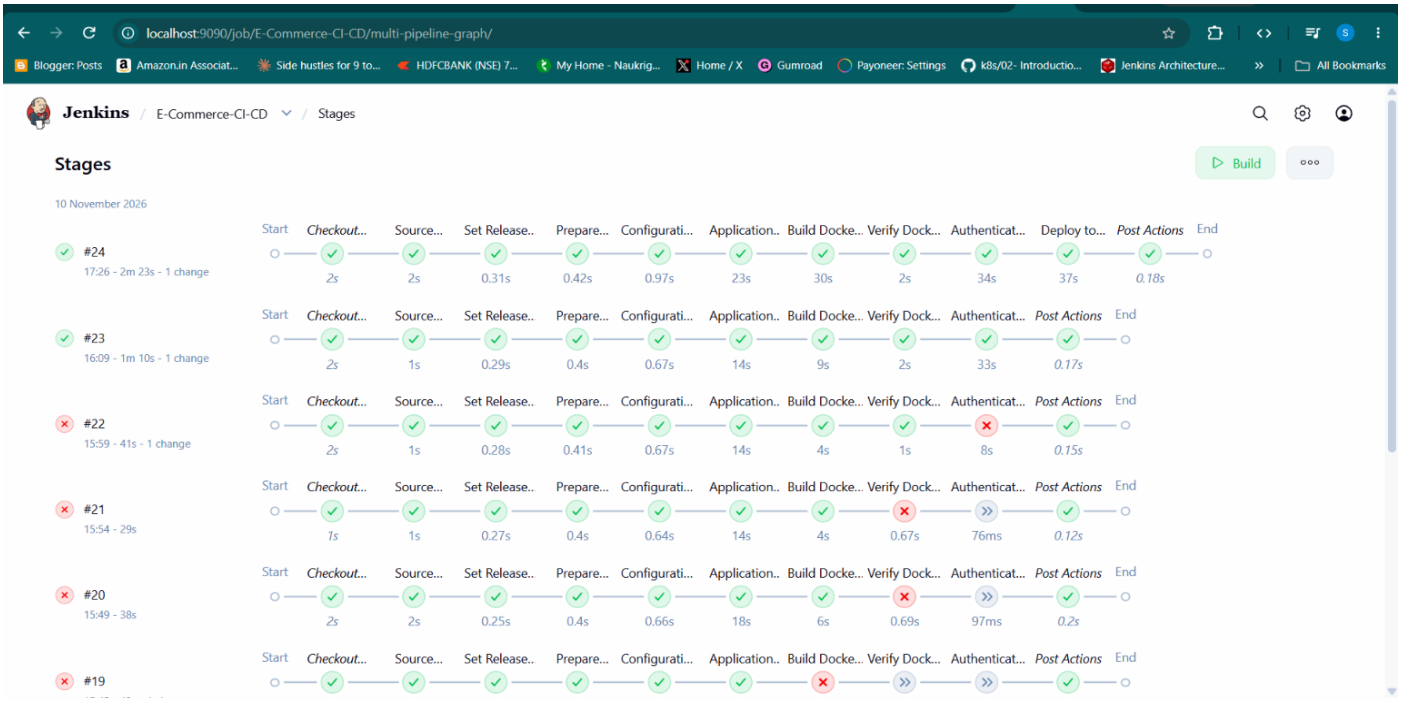


Figure 17. Jenkins CI/CD build process.

17. Deployment Automation Through AWS Systems Manager

The PowerShell script scripts/deploy-ssm.ps1 accepts an image tag, validates the tag format, creates an SSM Run Command request and polls the result. The remote Linux commands are executed on the EC2 host without requiring inbound SSH.

```
# Windows PowerShell – run deployment script manually for verification
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\deploy-ssm.ps1 -ImageTag
build-24
```

The remote deployment sequence records the previous release tag, writes the requested tag to image-tag.env, authenticates to ECR using the EC2 instance role, pulls the images, starts the Compose stack and checks the application response and backend health.

1. Validate the requested image tag.
2. Read the previous IMAGE_TAG value.
3. Write the new release tag to image-tag.env.
4. Authenticate to ECR using the instance role.
5. Pull the new images.
6. Run Docker Compose to recreate/update services.
7. Wait for service startup and health checks.
8. Verify the homepage and backend service health.
9. Report deployment success or attempt rollback.

18. Rollback and Recovery Test

A deliberate failure test was performed to demonstrate recovery. The deployment was asked to use a tag that did not exist in ECR: build-9999999. The image pull failed. The remote script's error handler attempted to restore the previous known-good release.

Step	Observed outcome
Previous release	build-24
Requested release	build-9999999
Failure point	ECR image pull for missing release image.
Recovery action	Restore image-tag.env to previous tag and pull previous images.
Recovered release	build-24
Post-recovery health	Backend health checks passed; containers were running.
Script result	ROLLBACK_COMPLETED: build-24

The SSM command itself was marked Failed because the requested new deployment failed. That status is expected for the failed release attempt; the important recovery evidence is that the rollback restored the previous application version and health checks passed.

Limit of the test This proves the specific missing-image deployment failure path. It does not guarantee recovery from host failure, corrupted state, invalid shell syntax or a failed rollback operation.

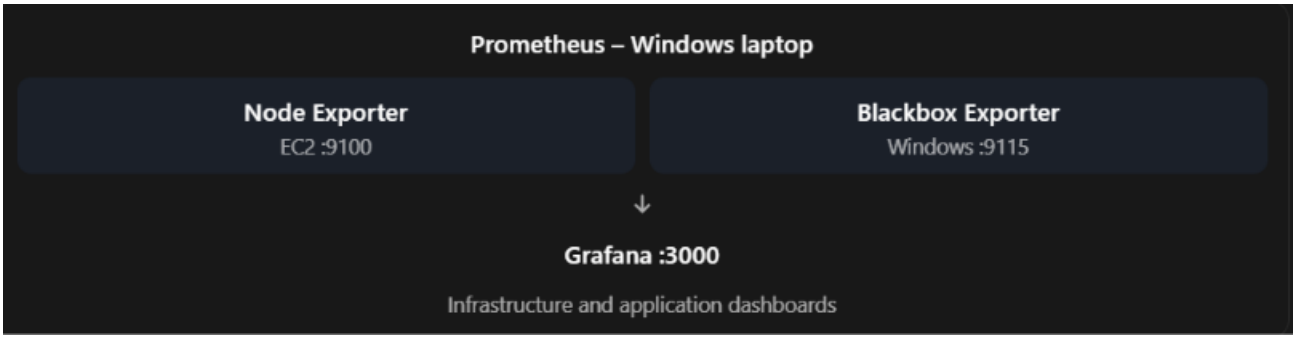
19. Monitoring Design – Why We Added Observability

Once the application was deployed, the next question was how to know whether it remained healthy. Monitoring was added in two layers: host-level metrics and an external application availability probe.

Tool	Role in ShopSphere
Node Exporter	Exports Linux host metrics from EC2 on port 9100.
Prometheus	Scrapes and stores metrics on the Windows workstation, port 9091.
Grafana	Connects to Prometheus and displays dashboards on port 3000.
Blackbox Exporter	Probes the public website through HTTP on port 9115.

Prometheus and Grafana were installed locally on Windows to avoid adding monitoring containers and memory overhead to the small t3.small EC2 instance. Jenkins already used port 9090, so Prometheus was configured to listen on 9091.

Important architecture detail The monitoring processes run on the Windows workstation, not on the EC2 instance. Node Exporter runs on EC2. The SSM tunnel connects the host metrics endpoint to local Prometheus.



20. Node Exporter – Linux Host Metrics

Node Exporter was installed on Ubuntu using the prometheus-node-exporter package. The service was enabled at boot and started with systemd. It exposes metrics at localhost:9100 on the EC2 host.

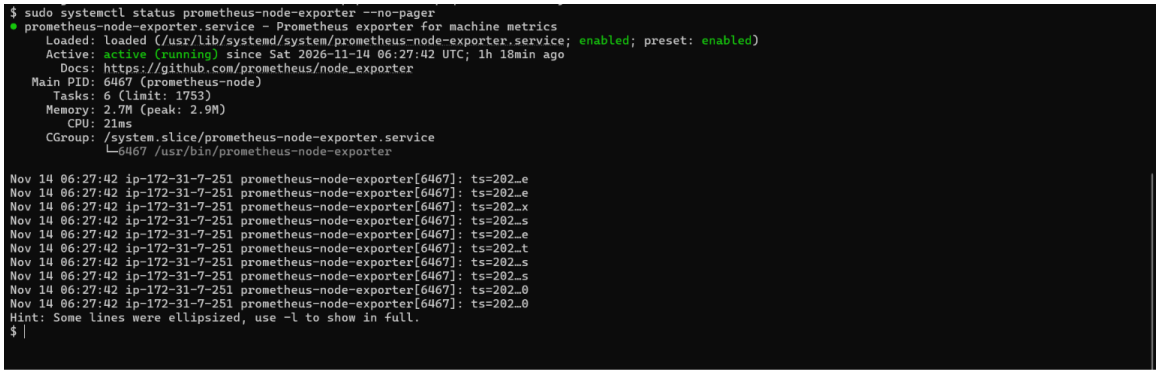


Figure 18. Node Exporter systemd service status on Ubuntu EC2.

```
# Ubuntu EC2 – enable and start Node Exporter
sudo systemctl enable --now prometheus-node-exporter

# Verify service state
sudo systemctl status prometheus-node-exporter --no-pager

# Read the metrics endpoint locally on EC2
curl -fsS http://localhost:9100/metrics | head -n 10
```

The endpoint provides metrics that can be used to calculate CPU utilization, available memory, filesystem usage, network traffic and system uptime.

Security note Port 9100 is not opened to the public internet. Prometheus reaches it through an SSM port-forwarding session.

21. Prometheus – Collection and Configuration

Prometheus was installed on Windows under C:\DevOps-Monitoring\prometheus. The executable used was Prometheus 3.16.0 for Windows AMD64. Its web interface and scrape service listened on 127.0.0.1:9091.

Setting	Value
Operating system	Windows workstation
Prometheus version	3.16.0 windows-amd64
Listen address	127.0.0.1:9091
Configuration	C:\DevOps-Monitoring\prometheus\prometheus.yml
Scrape interval	15 seconds
Node Exporter target	127.0.0.1:9100 (through SSM tunnel)

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "ec2-node-exporter"
    static_configs:
      - targets: ["127.0.0.1:9100"]
```

```
# Windows PowerShell – start Prometheus
& "C:\DevOps-Monitoring\prometheus\prometheus-3.16.0.windows-amd64\prometheus.exe" --
config.file="C:\DevOps-Monitoring\prometheus\prometheus.yml" --web.listen-address="127.0.0.1:9091"
```

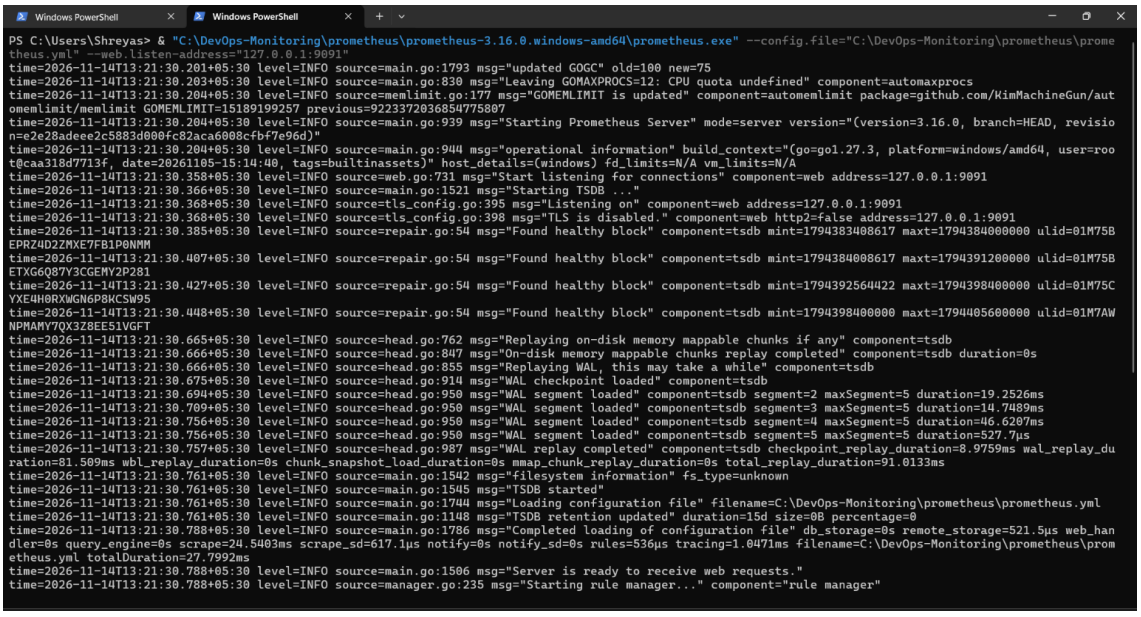


Figure 19. Prometheus process starting on Windows.

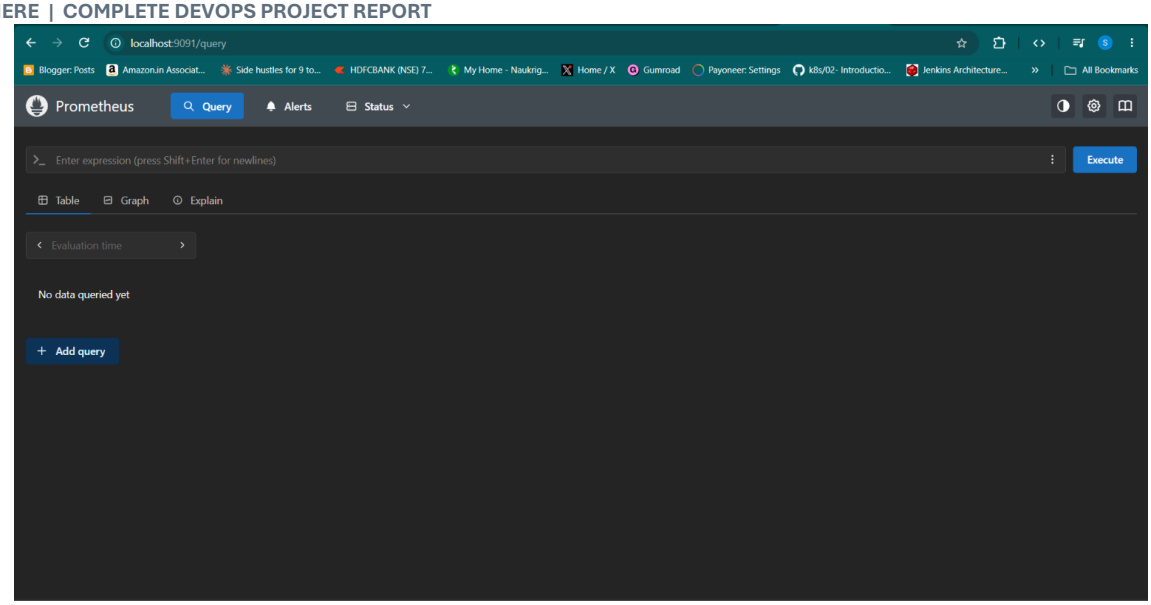


Figure 20. Prometheus web interface.

22. SSM Port Forwarding – Secure Metrics Connectivity

Because Node Exporter listens on the EC2 host, Prometheus on Windows needs a network path to reach it. Rather than adding a public inbound security group rule for port 9100, the project uses AWS SSM port forwarding.

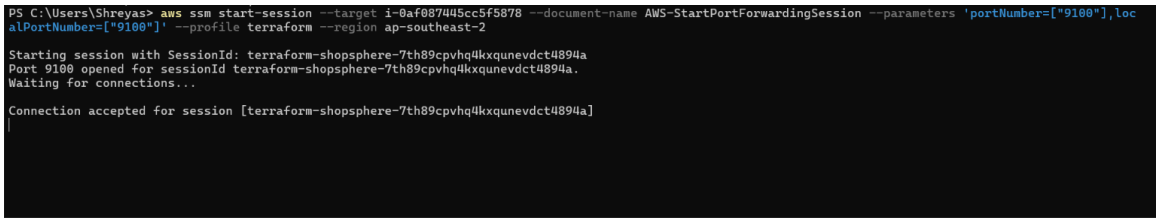


Figure 21. SSM port-forwarding session established from Windows.

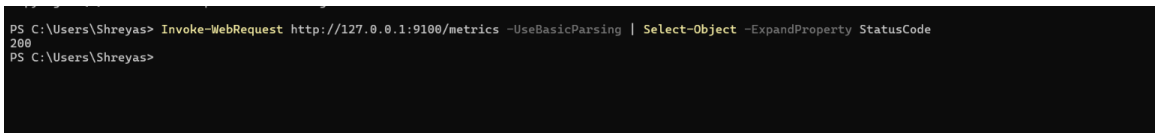


Figure 22. Local test of the forwarded Node Exporter endpoint.

```
# Windows PowerShell – test the local forwarded port
Test-NetConnection 127.0.0.1 -Port 9100
Invoke-WebRequest http://127.0.0.1:9100/metrics -UseBasicParsing
```

Keep the SSM port-forwarding terminal open while Prometheus is scraping Node Exporter. If the tunnel is closed, Prometheus reports the target as DOWN even when the Node Exporter service is healthy on EC2.

Troubleshooting clue A DOWN target at 127.0.0.1:9100 can mean the local tunnel is missing. First test the local port and confirm the SSM session is still active before changing AWS security groups.

23. Grafana – Dashboards and Data Source

Grafana was installed locally on Windows and configured to use Prometheus as its data source. The Prometheus URL entered in Grafana was http://localhost:9091. The imported Node Exporter Full dashboard provided host-level views.

Setting	Value
Grafana UI	http://localhost:3000
Prometheus data source	http://localhost:9091
Dashboard imported	Node Exporter Full (ID 1860)
Metrics visualized	CPU, memory, filesystem, network and uptime

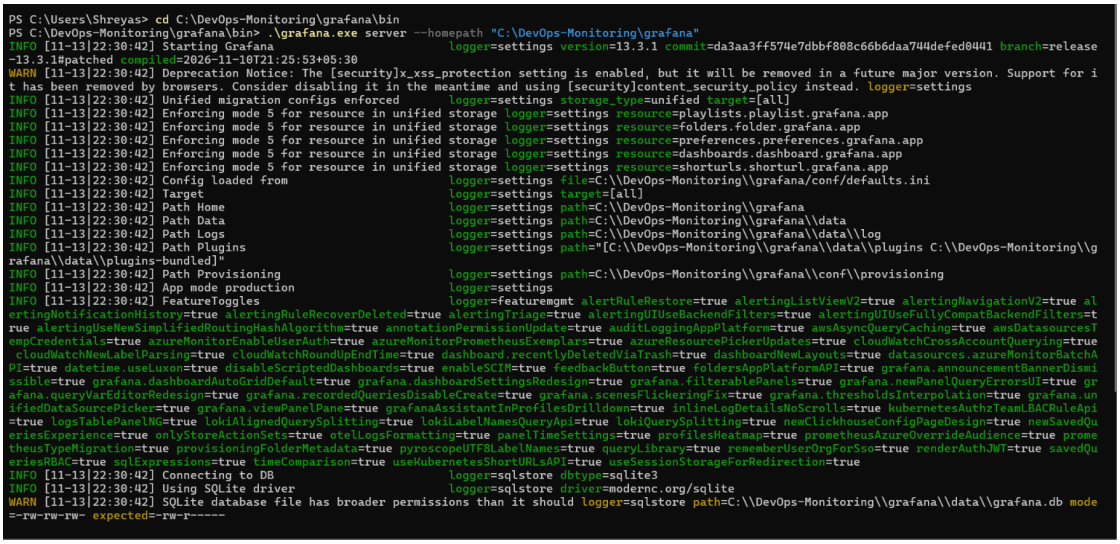


Figure 23. Grafana server starting on Windows.

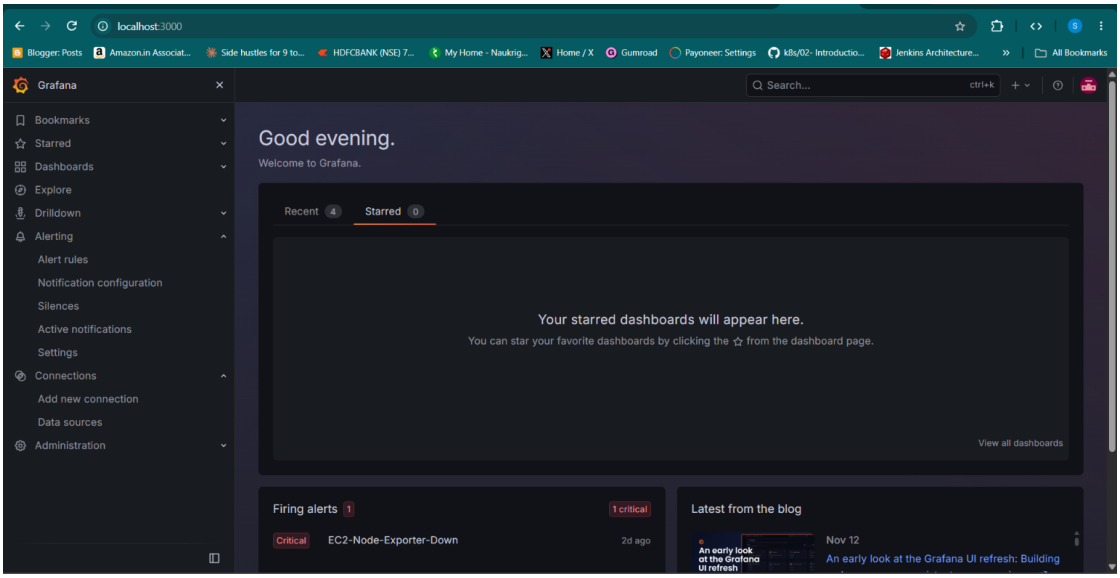


Figure 24. Grafana UI available on localhost:3000.

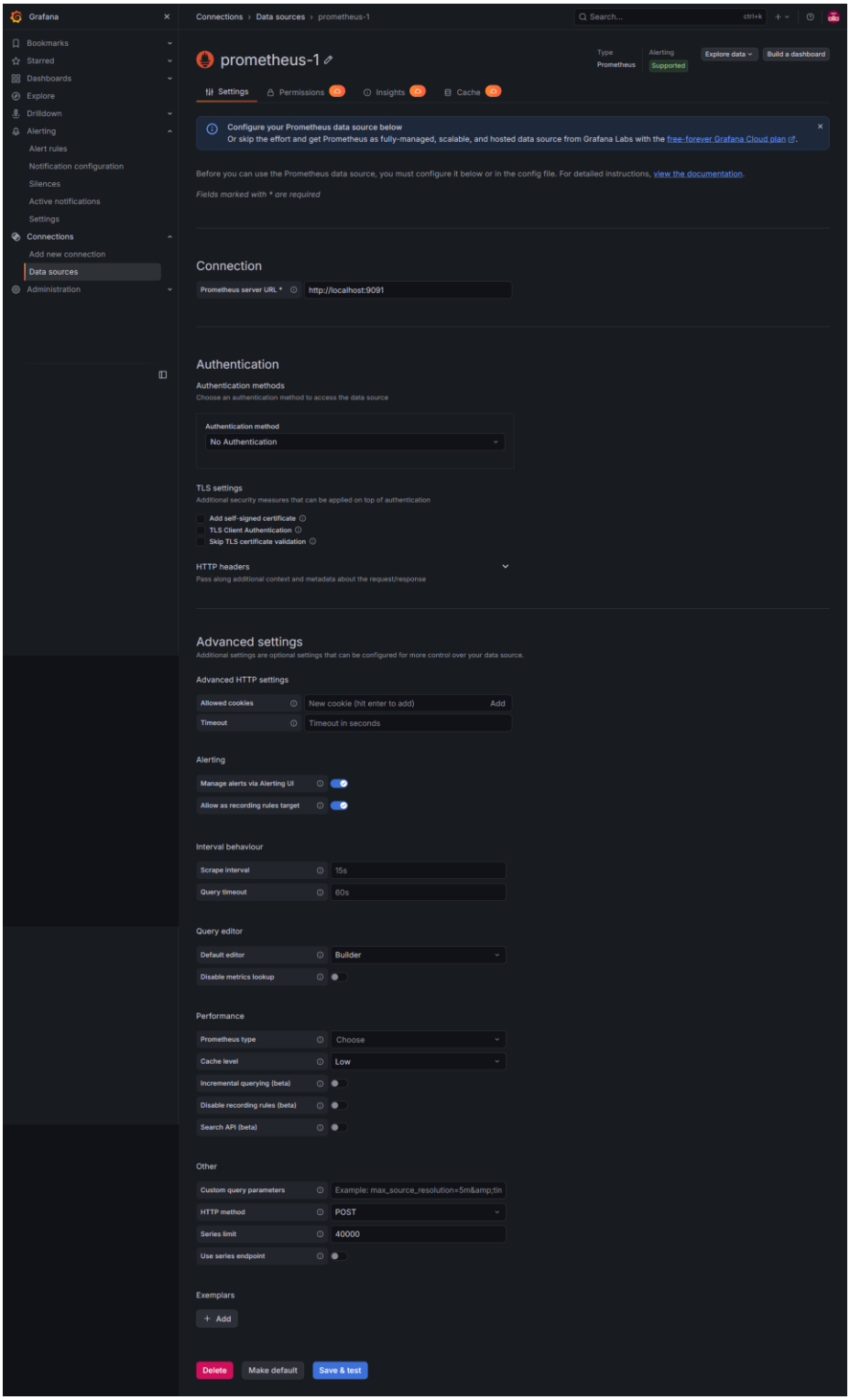


Figure 25. Grafana Prometheus data source configuration.

24. Grafana Host Dashboard and Alerting

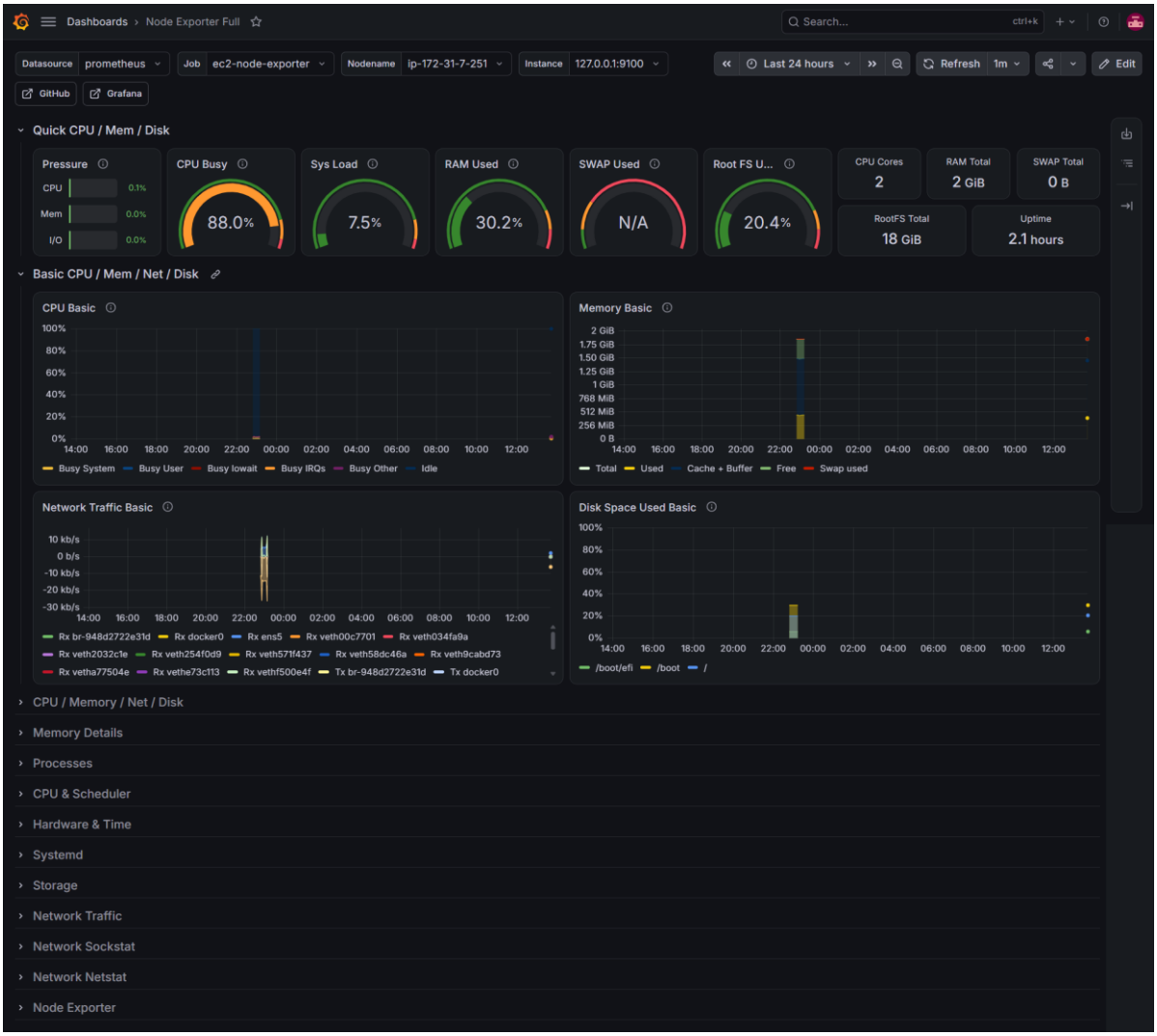


Figure 26. Node Exporter Full dashboard with host resource panels.

The imported dashboard made it possible to inspect CPU utilization, RAM use, filesystem and network metrics. A Grafana-managed alert rule named EC2-Node-Exporter-Down was configured using the Prometheus query up{job="ec2-node-exporter"}, with a condition below 1. The rule used a one-minute evaluation interval and one-minute pending period.

The alert labels included severity=critical, service=node-exporter and environment=production. The alert was observed in Firing during the controlled exporter failure test.

Notification status The alert rule was evaluated and its state was tested. A Grafana contact point was not configured in the evidence, so email or chat notification delivery should be considered pending.

25. Blackbox Exporter – Website Availability

Blackbox Exporter was added to test the ShopSphere public website from the Windows workstation. It listens on port 9115 and exposes a /probe endpoint. Prometheus asks Blackbox Exporter to probe the current public EC2 URL using the http_2xx module.

Setting	Value
Exporter version	0.29.1 windows-amd64
Listen port	9115
Probe module	http_2xx
Probe method	HTTP GET
Probe timeout	10 seconds
Target	Current public ShopSphere HTTP URL

```
modules:
  http_2xx:
    prober: http
    timeout: 10s
    http:
      method: GET
      preferred_ip_protocol: ip4
```

```
# Windows PowerShell – probe the current public endpoint
Invoke-WebRequest "http://127.0.0.1:9115/probe?target=http://CURRENT_EC2_IP/&module=http_2xx" -UseBasicParsing

# Inspect key results
$r.Content | Select-String "probe_success|probe_http_status_code|probe_duration_seconds"
```

A successful probe returned probe_success 1 and HTTP status 200. One observed successful probe completed in approximately 0.965 seconds. The public IP in the target must be updated if EC2 receives a different address after stop/start.

26. Prometheus Target Health and Failure Simulation

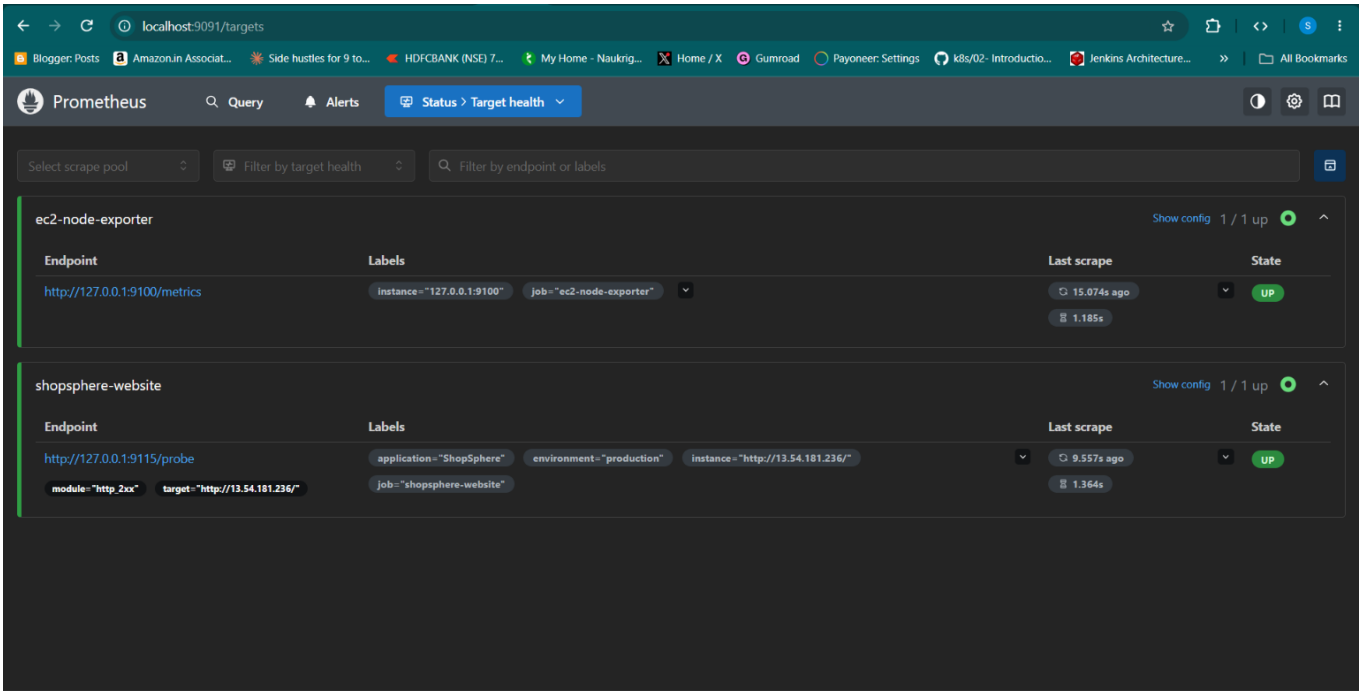


Figure 27. Prometheus target list showing the Node Exporter scrape target healthy.

The project also tested a monitoring failure by stopping Node Exporter. Prometheus then showed the Node Exporter target as DOWN, and the Grafana alert rule entered Firing. After Node Exporter was started again and the SSM tunnel was available, the target returned to UP.

Blackbox Exporter was independently able to probe the public ShopSphere endpoint. This distinction is useful: the website availability check and the EC2 host metrics check have different network paths and can fail for different reasons.

Operational lesson A monitoring system should be tested by deliberately creating a safe failure, observing the alert, restoring the service and confirming that the alert returns to normal.

27. Ports and Network Reference

Port	Component	Where it listens	Exposure / access
80	Nginx / ShopSphere	EC2 host to container	Public HTTP ingress in current SG.
3000	Grafana	Windows workstation	Local workstation UI.
9090	Jenkins	Windows workstation	Local workstation UI.
9091	Prometheus	Windows workstation, 127.0.0.1	Local only.
9100	Node Exporter	EC2 localhost	Accessed through SSM port forwarding; do not expose publicly.
9115	Blackbox Exporter	Windows workstation	Local probe endpoint.
3001	User service	Compose network	Internal container traffic.
3002	Product service	Compose network	Internal container traffic.
3003	Order service	Compose network	Internal container traffic.

The application backend ports are service-to-service ports inside the Docker Compose network. They do not need public security group ingress. The monitoring UI ports are local workstation ports.

28. Troubleshooting – Issues We Encountered

Issue	What caused it / how it appeared	What fixed it or what to check
ECR authorization failure	Jenkins IAM user lacked required ECR actions.	Add least-privilege ECR push permissions and rerun.
Docker push broken pipe	Docker Desktop proxy configuration interfered with registry upload.	Review Docker proxy and no-proxy settings; retest push.
Invalid AWS token	AWS CLI profile contained invalid credentials.	Reconfigure the correct CLI profile and region.
SSM JSON parsing	PowerShell changed inline JSON quoting.	Write JSON to a UTF-8 file and pass it using --cli-input-json.
SSM access denied	Wrong AWS identity used for Session Manager.	Use an identity authorized for StartSession.
Node Exporter target DOWN	Local 9100 port was not forwarded or tunnel was closed.	Reopen SSM port-forwarding session and test localhost:9100.
Blackbox executable not found	PowerShell was in parent directory; executable was in nested folder.	Change to extracted version folder before running executable.
Blackbox timeout	Probe target did not respond within timeout / old IP used.	Verify current EC2 public IP and test website directly.
Rollback triggered	Requested image tag did not exist in ECR.	Previous release tag restored; verify services and homepage.

These troubleshooting exercises are valuable project evidence because they show diagnosis and recovery, not just a successful final state.

29. Security, Reliability and Cost Notes

Security controls applied

- Use EC2 IAM instance role for ECR pulls and SSM rather than storing AWS access keys on the server.
- Use Jenkins Credentials for the CI/CD AWS credential.
- Keep production .env outside Git and restrict its file permissions.
- Do not expose Node Exporter port 9100 publicly; use SSM port forwarding.
- Keep Terraform state and provider working directory out of source control.

Reliability limitations

- One EC2 instance is a single point of failure.
- No load balancer or multi-AZ failover was implemented.
- The public IP may change after stop/start.
- Monitoring runs on the developer workstation and is unavailable if that workstation is off or disconnected.
- The Node Exporter Prometheus target depends on an active SSM tunnel.

Cost considerations

The solution intentionally uses one t3.small EC2 instance, Docker Compose, existing ECR repositories and local Prometheus/Grafana/Blackbox Exporter processes. This avoids a continuously running Kubernetes cluster or managed monitoring stack. Stopping EC2 reduces compute charges, but EBS volumes and stored ECR images can continue to incur charges.

30. Project Status – Completed, In Progress and Pending

Workstream	Status	Evidence / next action
EC2, IAM, security group and SSM	Completed	Instance and SSM session verified.
Terraform provisioning	Completed	init, validate, plan and apply completed.
ECR image repositories	Completed	Four repositories used for application images.
Docker Compose and Nginx	Completed	Five containers observed; backend health checks passed.
Jenkins CI/CD	Completed	Build, push and SSM deployment pipeline succeeded.
Deployment rollback	Completed / tested	Missing-tag test restored build-24.
Node Exporter	Completed	Service active and metrics endpoint responded.
Prometheus Node Exporter scrape	Completed	Target UP when SSM tunnel is active.
Grafana host dashboard	Completed	Node Exporter Full dashboard imported.
Grafana alert evaluation	Completed / tested	Firing and recovery observed; notification channel absent.
Blackbox Exporter	Completed	ShopSphere HTTP probe returned success and status 200.
Unified ShopSphere dashboard	In progress	Build one dashboard combining host and website availability.
Email/chat notifications	Pending	Configure contact point and test delivery.
Container metrics	Pending	Add cAdvisor or equivalent and dashboard panels.
Centralized logs	Pending	Add Loki/Promtail or another log platform.
HTTPS and custom domain	Future enhancement	Add TLS certificate and DNS.
High availability	Future enhancement	Consider load balancer, multiple instances and multi-AZ design.

31. Recommended Next Steps

The next steps should build on what is already working rather than introducing more infrastructure immediately.

10. Finish the unified Grafana dashboard with panels for EC2 CPU, memory, disk, network, Node Exporter availability, ShopSphere probe success, HTTP status and response time.
11. Configure a Grafana contact point and test a real notification to an approved email or chat destination.
12. Add container-level metrics only if the host has enough memory and disk headroom; monitor the t3.small resource budget.
13. Choose a centralized log solution and define retention before installing it.
14. Add HTTPS and a stable DNS name before treating the service as a public production deployment.
15. Move Terraform state to a secured remote backend if the project will be maintained collaboratively.
16. Add a documented release checklist and periodic rollback drill.

Keep the scope sensible Do not add EKS, a load balancer or extra managed services just to make the architecture look bigger. Add infrastructure only when a real availability, scale or operational requirement justifies it.

32. Operational Runbook – Starting the Project

This checklist is intended for the next time the environment is powered down and needs to be brought back online.

17. Start the existing EC2 instance in ap-southeast-2 and wait for the instance status checks to pass.
18. Wait until Systems Manager shows the managed instance as Online.
19. Open an SSM session to the EC2 instance using the approved AWS CLI profile.
20. On Ubuntu, check Docker: sudo systemctl is-active docker.
21. Go to /opt/shopsphere and inspect image-tag.env.
22. Run docker compose --env-file image-tag.env ps and confirm the five services.
23. If services are stopped, run docker compose --env-file image-tag.env up -d.
24. Check http://localhost/ from the EC2 host and confirm HTTP 200.
25. Check Node Exporter service status.
26. On Windows, start Prometheus, Grafana and Blackbox Exporter if needed.
27. Recreate the SSM port-forwarding session for port 9100.
28. Check Prometheus /targets and verify both expected jobs are UP.

33. Operational Runbook – Useful Commands

Windows PowerShell – inspect EC2 state and public IP

```
aws ec2 describe-instances --instance-ids i-0af087445cc5f5878 --profile terraform --region ap-southeast-2 --query
"Reservations[0].Instances[0].{State:State.Name,PublicIP:PublicIpAddress}" --output table
```

Ubuntu EC2 – application and Docker

```
sudo systemctl is-active docker
cd /opt/shopsphere
cat image-tag.env
docker compose --env-file image-tag.env ps
curl -sS -o /dev/null -w 'HTTP Status: %{http_code}\n' http://localhost/
```

Ubuntu EC2 – Node Exporter

```
sudo systemctl status prometheus-node-exporter --no-pager
curl -fsS http://localhost:9100/metrics | head -n 10
```

Windows PowerShell – monitoring checks

```
Test-NetConnection 127.0.0.1 -Port 9100
Invoke-WebRequest http://127.0.0.1:9100/metrics -UseBasicParsing
Invoke-WebRequest http://127.0.0.1:9115/metrics -UseBasicParsing
```

Command safety Run Ubuntu commands only inside the EC2 SSM shell. Run Windows PowerShell commands on the workstation. Do not paste shell commands into the wrong terminal.

34. Repository and Project Structure

The project maintained the local source repository on Windows. The high-level structure separated backend service Dockerfiles, frontend, Nginx, Compose, Jenkins automation, deployment scripts and Terraform definitions.

```
Microservice-application/
  services/
    user-service/
      Dockerfile
    product-service/
      Dockerfile
    order-service/
      Dockerfile
  frontend/
    Dockerfile
  nginx/
    nginx.conf
  docker-compose.yml
  .env.example
  .env                  # ignored; do not commit
  Jenkinsfile
  scripts/
    deploy-ssm.ps1
  terraform/
    providers.tf
    variables.tf
    main.tf
    iam.tf
    security-groups.tf
    ecr.tf
    outputs.tf
```



Figure 28. Project folder structure shown in the supplied source document.

The Terraform lock file should be committed so the provider version remains reproducible. Terraform state, secret environment files and local build artifacts should remain excluded from Git.

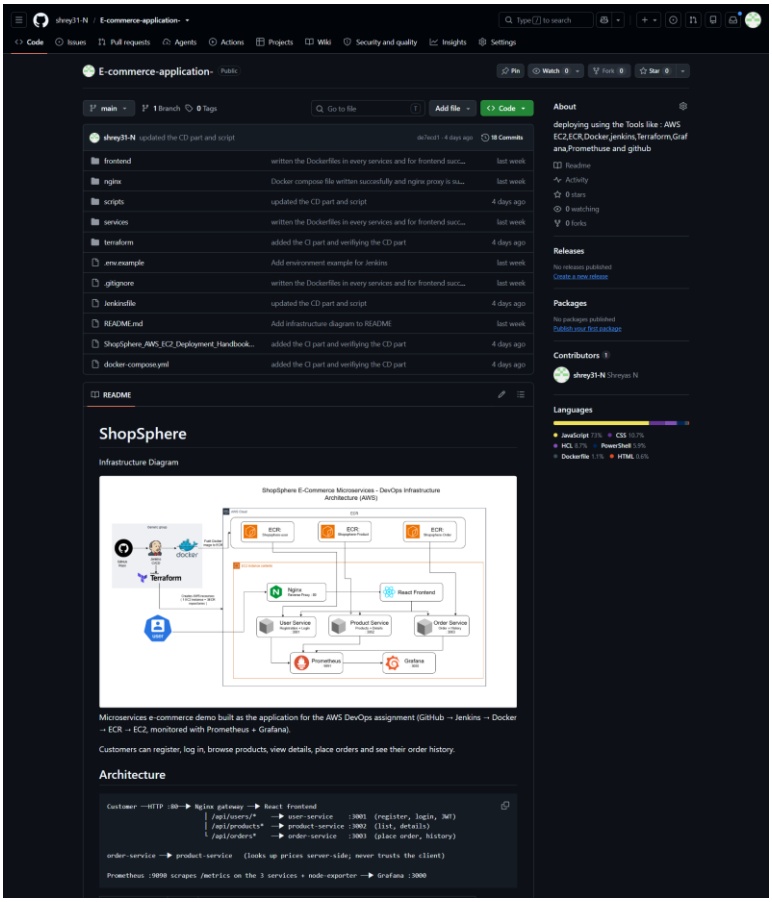


Figure 29. GitHub repository.

35. Final Summary and Learning Outcomes

This project brought together the main building blocks of a practical DevOps workflow. The application was packaged into containers, deployed on AWS, and served through Nginx. Terraform described the core infrastructure. Jenkins automated image creation and delivery. SSM provided remote operations without opening SSH. A rollback test demonstrated recovery to a known-good image tag.

Monitoring was extended from host metrics to website availability. Node Exporter exposed host metrics, Prometheus collected them, Grafana displayed them and evaluated an alert, and Blackbox Exporter checked the public website. The failure simulation helped confirm that the monitoring path could detect and recover from a service interruption.

The most important lesson is that DevOps is a combination of automation, repeatability, security, feedback and recovery. A successful deployment is only one part of the job. An engineer must also know how to verify it, diagnose it, roll it back and communicate its limitations.

Final status The core application deployment and monitoring checks described in this report were demonstrated. The unified dashboard, notification delivery, container metrics and centralized logging remain the next work items.

Appendix – Evidence source and image placement

Screenshots in this report were taken from the supplied 13-page ShopSphere document and repositioned beside the relevant implementation explanation. The original file showed EC2/SSM setup on pages 1–2, application user flows on pages 2–6, Node Exporter and Prometheus setup on pages 6–8, Grafana setup on pages 8–10, dashboard evidence on pages 11–12, and the architecture diagram on page 13.

The source screenshots were not used as decorative filler. Each image is placed next to the section it supports and captioned to explain what the reader should notice.